

Security Audit

DigiShares (RWA)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Status Definitions	16
Audit Findings	16
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The DigiShares team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

DigiShares is an end-to-end real-world asset (RWA) tokenization platform that enables businesses to digitize and manage ownership of assets—such as real estate, private equity, and funds—on blockchain infrastructure. The platform provides tools for fractionalizing assets into tradable digital securities, handling investor onboarding (KYC/AML), cap table management, dividend distribution, and secondary trading via integrated marketplaces or external exchanges. Designed for compliance and institutional use, DigiShares acts as a bridge between traditional finance and Web3 by automating back-office processes, increasing liquidity, and expanding global investor access to previously illiquid assets.

Project Name: DigiShares

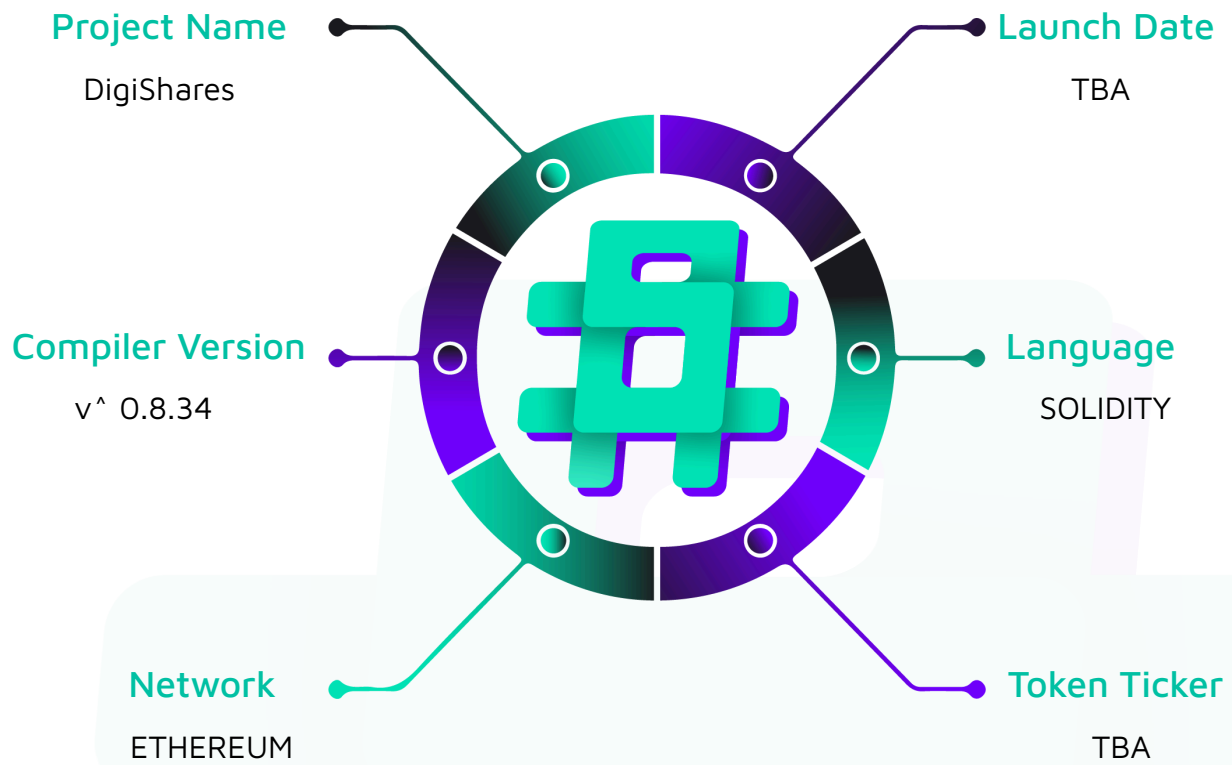
Project Type: RWA

Compiler Version: ^ 0.8.34

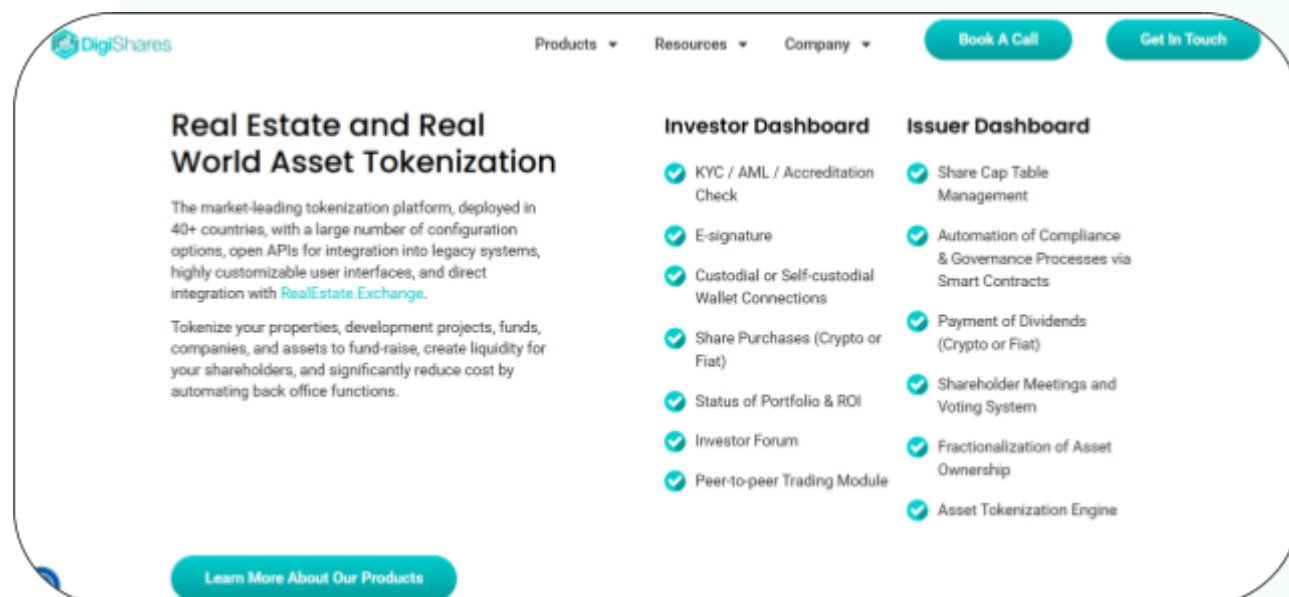
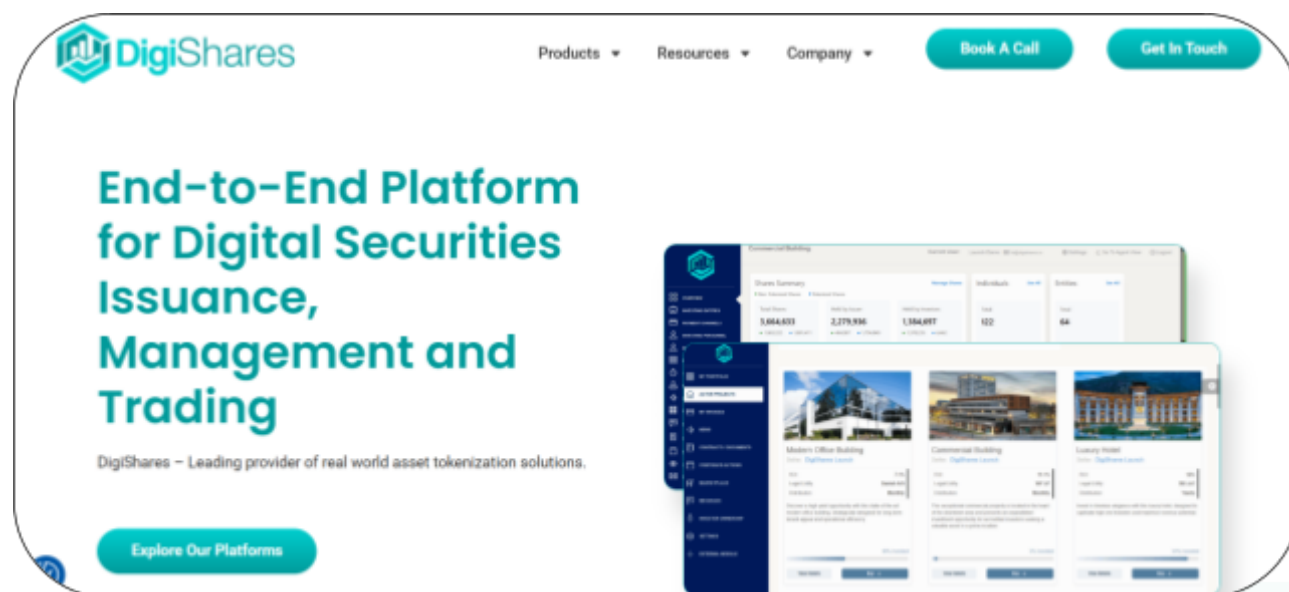
Website: <https://digishares.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the DigiShares project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	DigiShares Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	April, 2026
Contract 1	ABT20.sol
Contract 2	ABT20Basic.sol
Contract 3	ABT20Burn.sol
Contract 4	ABT20BurnPause.sol
Contract 5	ABT20Mint.sol
Contract 6	ABT20MintBurn.sol
Contract 7	ABT20MintBurnCap.sol
Contract 8	ABT20MintBurnPause.sol
Contract 9	ABT20MintCap.sol
Contract 10	ABT20MintCapPause.sol
Contract 11	ABT20MintPause.sol
Contract 12	ABT20Pause.sol

Audited GitHub Commit Hash	eef85508a345b6ad0299d5ab7ced54f017ed1014
Fix Review GitHub Commit Hash	34119b3e9d1ce3e829cbe36ec4256a00585c88d5



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Gas Optimisations

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ABT20.sol - Allows users to: - Library/Interface — callable helpers only (ERC20 + ERC20Permit + Multicall base with configurable decimals) - Allows admins to: - None	Contract achieves this functionality.
ABT20Basic.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - None	Contract achieves this functionality.
ABT20Burn.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Burn tokens from any address (BURNER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.
ABT20BurnPause.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall	Contract achieves this functionality.

- Allows admins to: - Burn tokens from any address (BURNER_ROLE) - Pause and unpause transfers (PAUSER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	
ABT20Mint.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Mint new tokens to any address (MINTER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.
ABT20MintBurn.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Mint new tokens to any address (MINTER_ROLE) - Burn tokens from any address (BURNER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.
ABT20MintBurnCap.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Mint tokens up to a fixed cap (MINTER_ROLE) - Burn tokens from any address (BURNER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.

ABT20MintBurnPause.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Mint new tokens to any address (MINTER_ROLE) - Burn tokens from any address (BURNER_ROLE) - Pause and unpause transfers (PAUSER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.
ABT20MintCap.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Mint tokens up to a fixed cap (MINTER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.
ABT20MintCapPause.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Mint tokens up to a fixed cap (MINTER_ROLE) - Pause and unpause transfers (PAUSER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.
ABT20MintPause.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals	Contract achieves this functionality.

- Batch calls via multicall - Allows admins to: - Mint new tokens to any address (MINTER_ROLE) - Pause and unpause transfers (PAUSER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	
ABT20Pause.sol - Allows users to: - Transfer and approve ERC20 tokens - Use EIP-2612 permit for gasless approvals - Batch calls via multicall - Allows admins to: - Pause and unpause transfers (PAUSER_ROLE) - Manage roles (DEFAULT_ADMIN_ROLE)	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the DigiShares project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the DigiShares project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

QA

[Q-01] Multiple Contracts - Missing Zero-Address Validation on Constructor Role Parameters

Description

Eleven AccessControl-bearing ABT20 variants accept role addresses (`defaultAdmin`, `minter`, `burner`, `pauser`) as constructor parameters without validating that the values are non-zero. If a deployer passes `address(0)` for any role due to a deployment script bug or misconfiguration, the role is silently granted to the zero address. No account can act as `address(0)`, so the affected role becomes permanently non-functional with no recovery path. Because the contracts are not upgradeable, the only remediation would be a full redeployment.

Recommendation

Add `require` guards at the top of each affected constructor before any role is

Granted:

```
constructor(..., address defaultAdmin, address minter, address burner, address pauser,
...) {
+   require(defaultAdmin != address(0), "ABT20: zero admin");
+   require(minter      != address(0), "ABT20: zero minter");
+   require(burner      != address(0), "ABT20: zero burner");
+   require(pauser      != address(0), "ABT20: zero pauser");
    _mint(initialHolder, initialSupply);
    _grantRole(DEFAULT_ADMIN_ROLE, defaultAdmin);    ...
}
```

Apply equivalent guards to all remaining variants.

Status

Resolved

[Q-02] Multiple Contracts - Single-Address Admin Without Timelock or Two-Step Transfer

Description

All 12 AccessControl-bearing ABT20 variants assign `DEFAULT\_ADMIN\_ROLE` to a single EOA with immediate, unrestricted authority to grant and revoke all protocol roles. Two critical safety mechanisms are absent: a timelock that delays role changes (giving stakeholders time to react to a compromised key) and a two-step pending-then-accept pattern for `DEFAULT\_ADMIN\_ROLE` transfer (preventing permanent loss of admin control from a typo in the target address). All variants inherit `AccessControlEnumerable` rather than the hardened `AccessControlDefaultAdminRules` extension.

Recommendation

Replace `AccessControlEnumerable` with `AccessControlDefaultAdminRules` across all 12 variants:

```
- import {AccessControlEnumerable} from
-     "@openzeppelin/contracts/access/extensions/AccessControlEnumerable.sol";
-
- contract ABT20Burn is ABT20, AccessControlEnumerable {
-     constructor(..., address defaultAdmin, address burner) ABT20(...) {
+ import {AccessControlDefaultAdminRules} from
+     "@openzeppelin/contracts/access/extensions/AccessControlDefaultAdminRules.sol";
+
+ contract ABT20Burn is ABT20, AccessControlDefaultAdminRules {
+     constructor(..., uint48 adminDelay, address defaultAdmin, address burner)
+         ABT20(...)
+         AccessControlDefaultAdminRules(adminDelay, defaultAdmin)
+     {
        _mint(initialHolder, initialSupply);
-         _grantRole(DEFAULT_ADMIN_ROLE, defaultAdmin);
        _grantRole(BURNER_ROLE, burner);
```

```
}  
}
```

If replacing the base class is not immediately feasible, deploy admin role management through an external `TimeLockController` and transfer `DEFAULT_ADMIN_ROLE` to it, enforcing delays at the governance layer.

Status

Acknowledged — The team retains `AccessControlEnumerable` to preserve role enumeration.

GAS

[G-01] ABT20#decimals - Gas Overhead from Non-Immutable `\_decimals` Storage Variable

Description

The `\_decimals` variable in `ABT20` is declared as a mutable `uint8 private` storage slot despite being written exactly once at construction and never modified afterward. Every external call to `decimals()` therefore performs an unnecessary SLOAD. Declaring the variable `immutable` encodes the value directly into the deployed bytecode, eliminating the storage read entirely. All 13 in-scope contracts inherit `ABT20`, so the overhead applies to every deployed variant.

Recommendation

Replace the mutable declaration with `immutable`:

```
- uint8 private _decimals;
+ uint8 private immutable _decimals;

constructor(string memory name, string memory symbol, uint8 decimals_)
    ERC20(name, symbol)
    ERC20Permit(name)
{
    _decimals = decimals_;
}
```

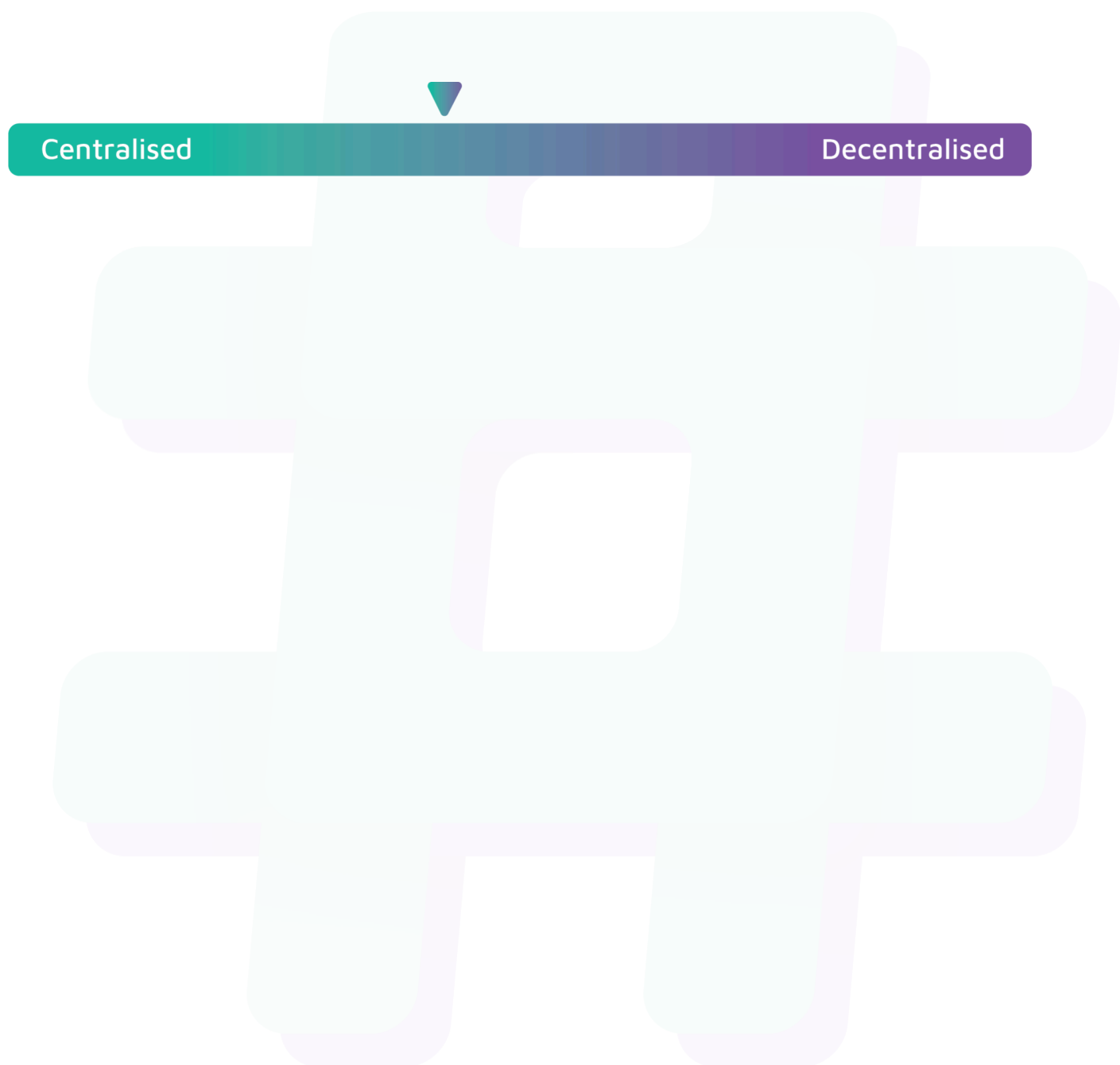
Status

Resolved

Centralisation

The DigiShares project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the DigiShares project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.



Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.